

Available online at www.sciencedirect.com

SCIENCE @ DIRECT®

Information Processing Letters ••• (••••) •••–•••

**Information
Processing
Letters**
www.elsevier.com/locate/ipl

Speeding up transposition-invariant string matching

 Sebastian Deorowicz¹
Silesian University of Technology, Institute of Computer Science, Akademicka 16, 44-100 Gliwice, Poland

Received 4 March 2005; received in revised form 25 January 2006; accepted 30 April 2006

Communicated by A. Moffat

Abstract

Finding the longest common subsequence (LCS) of two given sequences $A = a_0a_1 \dots a_{m-1}$ and $B = b_0b_1 \dots b_{n-1}$ is an important and well studied problem. We consider its generalization, transposition-invariant LCS (LCTS), which has recently arisen in the field of music information retrieval. In LCTS, we look for the LCS between the sequences $A + t = (a_0 + t)(a_1 + t) \dots (a_{m-1} + t)$ and B where t is any integer. We introduce a family of algorithms (motivated by the Hunt–Szymanski scheme for LCS), improving the currently best known complexity from $O(mn \log \log \sigma)$ to $O(D \log \log \sigma + mn)$, where σ is the alphabet size and $D \leq mn$ is the total number of dominant matches for all transpositions. Then, we demonstrate experimentally that some of our algorithms outperform the best ones from literature.

© 2006 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Information retrieval; Longest common transposition-invariant subsequence; Longest common subsequence; Transposition invariance

1. Introduction

One of the classical measures of similarity of two sequences is the length of their longest common subsequence (LCS). A subsequence of a sequence is obtained by deleting zero or more symbols and the problem is to find the longest sequence which is a subsequence of both sequences. This similarity measure has applications in many areas, e.g., DNA and protein analysis, text information retrieval, file comparing [5].

Recently it was also used in music information retrieval (MIR) for melodies stored in MIDI format,

which are described by pitch values and durations. It appeared, however, that the classical LCS is not well suited for MIR, since there are substantial differences between music and text: (i) to determine the similarity of melodies we can often ignore durations; (ii) the same melody is perceived when the pitch values are shifted by some amount. Therefore, a transposition-invariant version of LCS, called LCTS, for the pitch values is more appropriate [3,9,10]. The LCTS finds applications also in time series comparison, image comparison, and other areas [14].

2. Definitions and problem statement

The sequences $A = a_0a_1 \dots a_{m-1}$ and $B = b_0b_1 \dots b_{n-1}$ are over Σ , where $\Sigma \subset \mathbb{Z}$, called an *alphabet*, is a finite subset of integers. We assume here, however, for

E-mail address: Sebastian.Deorowicz@polsl.pl (S. Deorowicz).

URL: <http://www-zo.iinf.polsl.gliwice.pl/~sdeor>.

¹ Supported by Silesian University of Technology Research Project BK-239/RAu2/2005.

simplicity $\Sigma = \{0, \dots, \sigma\}$. A sequence is a *subsequence* of X , when it can be obtained from X by deleting zero or more symbols. A sequence is the *longest common subsequence* (LCS) of A and B when it is a subsequence of both A and B and has the largest possible length. (There may be many longest common subsequences since only the length of the LCS is unique.) A *transposed copy* of the sequence A , denoted as $A + t$, for some $t \in \mathbb{Z}$ is $(a_0 + t)(a_1 + t) \dots (a_{m-1} + t)$. The *longest common transposition-invariant subsequence* (LCTS) of A and B is the longest common subsequence of B and $A + t$ for any $t \in [-\sigma, \sigma]$. We assume, without loss of generality that $m \leq n$. For simplicity of presentation, we denote $x_0 x_1 \dots x_k$ by X_k . When $a_i = b_j$ we say we have a *match* for a pair (i, j) . If (i, j) is a match and the LCS length for A_i and B_j is k , then k is the *rank* of (i, j) . The total number of matches between A and B is denoted by r .

3. Background and related work

The easiest way to calculate the LCTS is to run any classical LCS algorithm for all possible values of t (see [1,13] for a survey of LCS algorithms). It means, however, that the time complexity is $O(\sigma)$ times greater than the complexity of the base algorithm. There are also algorithms specialized for LCTS, nevertheless, their origins are often in the LCS field, so we will discuss here some of the important concepts of the LCS solving algorithms.

The classical algorithm for the LCS employs dynamic programming:

$$L(i, j) = \begin{cases} 0 & \text{for } i = -1 \text{ or } j = -1, \\ \max(L(i-1, j), L(i, j-1)) & \text{for } i, j \geq 0, a_i \neq b_j, \\ L(i-1, j-1) + 1 & \text{for } i, j \geq 0, a_i = b_j. \end{cases}$$

Each cell of the L matrix contains the length of the LCS for the prefixes of the sequences, i.e., $L(i, j) = |\text{LCS}(A_i, B_j)|$. This simple algorithm uses $O(mn)$ time and space. Most of the more sophisticated methods for the problem exploit the fact that it is sparse and it is unnecessary to compute all the matrix cells, since important changes to L occur only in matching cells (see last line of the recurrence above).

To show the idea of these algorithms we need to define some other terms. We say that the match (i', j') *precedes* the match (i, j) iff $i' < i$ and $j' < j$. The match (i, j) is *k-dominant* if it has rank k and there is no other match (i', j') of rank k such that $i' \leq i$ and $j' \leq j$. A *chain* is a sequence of matches, each one preceding

the next. An *antichain* is a set of matches where none of them precedes any other. To solve the LCS problem it suffices to find the longest possible chain or equivalently to partition all the matches into a minimal number of antichains. The algorithm by Hunt and Szymanski [7] solves the LCS in $O((n+r) \log n)$ time. It finds the minimal antichain decomposition processing the matrix row-wise and incrementally extending antichains. There are also some algorithms faster than $O(mn)$ in the worst case, e.g., Crochemore's et al. $O(mn/\log m)$ [4] and Hyrö's $O(\lceil n/w \rceil m)$ [6] algorithms.

Now we are ready to come back to the LCTS problem. There are several algorithms specialized for this problem [8,12]. Currently, best worst-case complexity for LCTS is $O(mn \log \log \sigma)$ [14]. Unfortunately, the theoretically best algorithms are rather slow in practice, which can be seen in the experimental section. In this paper, we introduce a family of algorithms and compare them to:

- **BBB**—a binary branch and bound method [8] of complexity $O((mn + \log \sigma)\sigma)$ in the worst case and $O((mn + \log \log \sigma) \log \sigma)$ in the best case,
- **CDP**—a classical dynamic programming repeated for all $t \in [-\sigma, \sigma]$ of complexity $O(mn\sigma)$ [10],
- **KBB**—a k -ary branch and bound method [8] of complexity $O((mn + \log(\sigma k/(k-1)))\sigma k/(k-1))$ in the worst case and $O((mn + \log(k \log_k \sigma))k \times \log_k \sigma)$ in the best case (we used $k = 3$ in our experiments, following [8] where the authors found that this value is the best in their experiments; the choice of k was also confirmed in our preliminary experiments),
- **SDP**—a sparse dynamic programming [11] of complexity $O(mn \log m)$,
- **YBP**—a bit-parallel algorithm [2] of complexity $O(mn \lceil \sigma/w \rceil)$, where w is the machine word size (in bits),
- **HBP**—a bit-parallel LCS algorithm [6] repeated for all possible t values of complexity $O(\lceil n/w \rceil m\sigma)$,
- **NGMD**—a recent algorithm [14] of complexity $O(mn \log \log \sigma)$.

The first of our algorithms, our-1, improves the best known time complexity to $O(D \log \log \sigma + mn)$, where $D \leq mn$ is the total number of dominant matches for all the transpositions, however is rather slow in practice. The others are of a worse time complexity, but are much faster and outperform the fastest known algorithms more than two times over MIDI sequences.

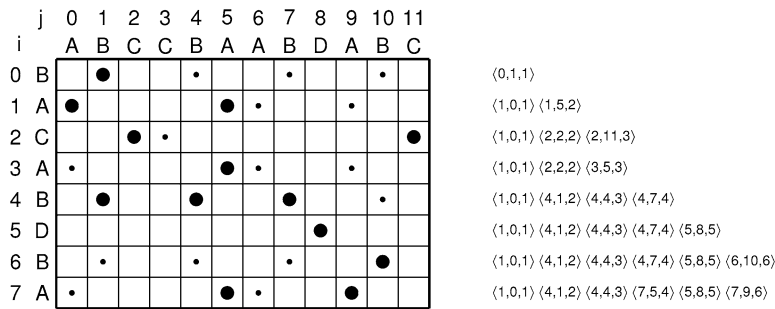


Fig. 1. Example of the algorithm in work. The left side shows the matrix with marked matches, while the right side presents the contents of the E data structure after processing each row.

4. Our approach

The idea of our method is to compute the LCS for all the transpositions using the sparsity of the LCS problem. Therefore, we will show how we find $|\text{LCS}(A, B)|$ and then we will discuss how this translates to the LCTS problem.

We compute the matrix L row-wise, i.e., we find the length of the longest common subsequence for A_i and B , for growing i . (The idea comes from Hunt and Szymanski [7], but we use a more efficient data structures to determine ranks of the matches.) We consider only matches so the total number of visited cells is r . When computing the i th row, we assume we know the ranks of all the matches for the rows $0, 1, \dots, i-1$. For the i th row² we move from left to the right (i.e., the column index grows from 0 to $n-1$). For each match (i, j) we compute its rank. To do this, we need to know the value of $L(i-1, j-1)$. It is obvious that it is the highest rank of all such matches (i', j') that $0 \leq i' < i$ and $0 \leq j' < j$.

To find the highest-rank match stored in the matrix $L[0..i-1, 0..j-1]$ it suffices to know the leftmost matches for each existing rank in $L[0..i-1, 0..n-1]$. Therefore we maintain a data structure E storing the triples $\langle \text{row_idx}, \text{col_idx}, \text{rank} \rangle$ for such matches. Notice that these are dominant matches and ranks in E grow by 1, since any k -dominant match must be preceded by some $(k-1)$ -dominant match. Moreover, the leftmost column indexes are increasing on rank .

When moving along the i th row, we visit matches with growing column indexes. For each match (i, j) , we look for a match with column index j in E . If it is present, the rank k of the current match must be the same, since any match with lower column index has

a lower rank. If it is not present, we look for the entry in E with the least col_idx value greater than j . It may happen that there is no such an entry in E , which immediately implies that the current match has higher rank than all matches seen so far and therefore the triple $\langle i, j, k+1 \rangle$, where k is the highest rank stored in E , is appended to E . In the other case, we have an entry $\langle i', j', k \rangle$. This triple is replaced by $\langle i, j, k \rangle$, since the current match is now the leftmost match of rank k seen so far (the match with rank $k-1$ must precede both (i', j') and (i, j)). We then skip all the matches in the current row for which the column index is not higher than j' , since all them have the same rank k . After processing the i th row, we do the same for the successive rows. In the end, the highest rank stored in E denotes the $\text{LCS}(A, B)$ length.

We could resign from checking whether there is a match with the column index j in E and the algorithm will remain correct. We, however, leave this operation since it has some influence on the time complexity, which will be discussed later.

Let us take a look at Fig. 1 for an example. After processing row 2 (we start counting rows from 0), E stores triples $\langle 1, 0, 1 \rangle, \langle 2, 2, 2 \rangle, \langle 2, 11, 3 \rangle$ (the matches of ranks 1, 2, 3 with lowest column indexes). Now, we process row 3. We see a match $(3, 0)$, but we already have match for column 0 in E (therefore, it is marked with a small dot) so we skip it. The next match, $(3, 5)$, is processed because we actually have no match for column 5 in E (we mark this using a big dot). We look in E for the match with lowest column index greater than 5 finding $\langle 2, 11, 3 \rangle$, and then we replace $\langle 2, 11, 3 \rangle$ in E by $\langle 3, 5, 3 \rangle$. We skip other matches in the current row and columns not higher than 11, that is, matches $(3, 6)$ and $(3, 9)$.

The ranks in E grow by 1, so it suffices to store only the pairs $\langle \text{row_idx}, \text{col_idx} \rangle$ ordered by col_idx and to compute, if necessary, the rank as the position of the

² Throughout the paper we use indexes for rows, columns, bits, trees, etc. starting from 0.

pair in E . Moreover, row indexes are also unimportant if we only want to know the $LCTS(A, B)$ length.

Let us now consider the time complexity of the method. At this moment we neglect the time necessary to find the set of matches between A and B . (We will discuss this issue later.) Now we assume that we know the r matches and for each match (i, j) we do a few operations on the E data structure. At most once we: look for the entry with column index j , look for the match with lowest column index greater than j , remove a match, insert a match. There are several candidates for E . We can use a balanced binary search tree, which leads to time $O(\log l)$ for each of these operations, so the total complexity would be $O(r \log l)$, where l is the LCS length. Since the matches in E are ordered by column index, which is from the range $[0, n - 1]$, we can also use a van Emde Boas tree [15,16], which offers the complexity $O(\log \log v)$ of necessary operations, where v is the range of the stored values, so in our case it would lead to the total complexity $O(r \log \log n)$.

We can also exploit the fact that for each row i we look for the monotonically growing column indexes. We use a family of $\lceil n/e \rceil$ van Emde Boas trees, each of size $e \leq n$ covering the possible column indexes range $[0, n - 1]$, i.e., the 0th tree stores values from $[0, e - 1]$, the 1st tree stores values from $[e, 2e - 1]$, and so on. (We assume $e \leq n$ since when $e > n$ there is only one tree.) When we insert, remove, or look for the column index j we use the $\lfloor j/e \rfloor$ th tree and each of these operations consumes $O(\log \log e)$ time. Looking for the lowest column index greater than j (finding a successor) is a bit more complex, because when the $\lfloor j/e \rfloor$ th tree does not contain such a value, we need to search the successive trees. Fortunately, within a single row, we always look for the monotonically growing successors. Therefore, we only once per row decide that the current tree does not contain the necessary value and we need to switch to the next tree. The time to find the successor in a tree is $O(\log \log e)$, and the additional time of unsuccessful searches is $O(n/e)$ per row. The total complexity for such a data structure is $O(r \log \log e + mn/e)$.

The last thing we can do is to improve the complexity of looking for the entry with column index j . We mentioned that we can resign from this operation and the algorithm will remain correct, but the time complexity also will remain unchanged. We can, however, accompany each van Emde Boas tree with a bit vector of size e , telling which values are stored in the tree. Thanks to this vector, the check operation can be done in a constant time and the cost of updating the vector by the insert and remove operations is also constant. Let us notice that the most frequent operation on E , executed

once for at most all the matches, is to check whether the column index of the match is in E . The other operations for a match are only executed if the match is dominant. This way we can improve the time complexity of the algorithm to $O(d \log \log e + r + mn/e)$, where d is the number of dominant matches.

Now we switch to the LCTS problem. We can do this in two alternative ways. The first one is to solve the $LCS(A + t, B)$ for all $t \in [-\sigma, \sigma]$. In this case, we run the described algorithm $\Theta(\sigma)$ times and the total number of matches is mn . Therefore, we obtain an algorithm for the LCTS problem of complexity $O(D \log \log e + mn + mn\sigma/e)$, where D is the total number of dominant matches for all the transpositions, which is obviously minimized for $e = \Theta(\sigma)$, yielding the total complexity of $O(D \log \log \sigma + mn)$.

We, however, should remember that some effort is necessary to prepare the sets of matches between $A + t$ and B . We can create a list for each alphabet symbol storing the positions of its appearance in B in time $O(n + \sigma)$ for the whole LCTS algorithm. Then, each time we move through the matrix rows we always have the positions of the matches ready. Because we traverse the matrix for each possible value of t , we need the additional time $O(m\sigma)$ for visiting the rows to check whether there are some matches in them or not.³ When $\sigma = O(n)$, the complexity obtained in the above paragraph is true, but otherwise we should write it as $O(D \log \log \sigma + m(n + \sigma))$. The space complexity of the algorithm is especially attractive because we need only the $O(\sigma)$ space for each of the $O(n/\sigma)$ van Emde Boas trees and $O(n + \sigma)$ space for other variables, so the total space consumption is $O(n + \sigma)$.

The second way of moving from the LCS to the LCTS problem is to assume that we traverse the matrix only one time and we visit all the cells, since for each of them we have a match for $A + t$ and B for some t . In other words, we do $O(\sigma)$ LCS algorithms in parallel. In this case, there is no additional time of preparing the sets of matches and there is no term $O(m\sigma)$ since we traverse the matrix only once. The time complexity is in this case $O(D \log \log \sigma + mn)$, while the space complexity grows to $O(n\sigma)$, because we need $O(\sigma)$ families of van Emde Boas trees, each for all the possible t values.

³ We can also prepare, in time $O(mn)$, the lists of matches for all possible transpositions and use them in the main algorithm, so it would not be necessary to pay the $O(m\sigma)$ time, but the lists consume $O(mn)$ space and our goal in this paragraph is to provide a space efficient version of the algorithm.

```

    { A[0..m - 1], B[0..n - 1] }
01 for t ← -σ to σ do
02   L[t] ← 0; N[t] ← -1;
03   for j ← 0 to (n + σ - 1) div σ - 1 do E[t][j].init(σ);
04   for i ← 0 to m - 1 do
05     for j ← 0 to n - 1 do
06       t ← B[j] - A[i];
07       if j + i × n > N[t] and not E[t][j div σ].check(j mod σ) then
08         s ← E[t][j div σ].findsucc(j mod σ);
09         if s < 0 then
10           for k ← j div σ + 1 to (n + σ - 1) div σ - 1 do
11             if E[t][k].min() ≥ 0 then
12               s ← k × σ + E[t][k].min(); break;
13           else s ← s + j - j mod σ;
14           E[t][j div σ].insert(j mod σ);
15           if s ≥ 0 then N[t] ← s + i × n; E[t][s div σ].remove(s mod σ);
16           else N[t] ← n - 1 + i × n; L[t] ← L[t] + 1;
17   lcts ← L[-σ];
18   for i ← -σ + 1 to σ do lcts ← max(lcts, L[i]);

```

Fig. 2. An $O(D \log \log \sigma + mn)$ algorithm for LCTS. (The operations `findsucc` and `min` return -1 when there is no successor or minimal value, respectively.)

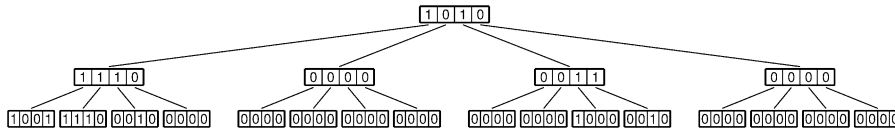


Fig. 3. Sample tree of arity 4 storing integers from the range $[0, 63]$ (the integers 0, 3, 4, 5, 6, 10, 40, 46 are stored in leaves).

The pseudo code of the above algorithm solving the LCTS problem is presented in Fig. 2. We assumed here that $e = \sigma$ for simplicity. During the algorithm we compute the LCS lengths for all the possible values of t and store them in the array $L[-\sigma.. \sigma]$. The array $N[-\sigma.. \sigma]$ stores the column indexes of the recently found matches in E , which were updated. In fact, we store in it column indexes plus the row indexes multiplied by the row length to avoid the necessity of clearing the array after processing each row. This way the new values $j + i \times n$ in next rows will be always larger than the values in N .

The presented algorithm improves the time complexity obtained recently [14] and is much simpler. Since van Emde Boas trees are slow in practice, we propose to replace them with some other data structures. Convenient candidates are w -ary trees, where w is the machine word size (in bits). Each inner node stores up to w bits; a bit is set iff the corresponding child node is non-empty. Data are stored in leaves; a bit is set iff the integer is in the tree (see Fig. 3).

Assuming we can count the number of leading zeroes in an integer in $O(1)$ time (which is true for modern processors) we can implement each of the necessary op-

erations on the tree to work in time $O(\lceil \log \sigma / \log w \rceil)$.⁴ To check whether x is in the tree it suffices to check the $(x \bmod w)$ th bit in the $\lfloor x/w \rfloor$ th leaf. Inserting or removing a value needs to set or reset the corresponding bit in the leaf and update the nodes upward the tree. The successor operation for x is more tricky. It is done by finding whether a bit higher than $(x \bmod w)$ is set in the leaf $\lfloor x/w \rfloor$. If not, we move upward the tree and follow analogously until we find such a bit, then we move downward and take a minimum of the subtree. All these operations need $O(\log v / \log w)$ time, where v is the range of stored values. Setting $v = \Theta(\sigma)$ leads to the $O(D \lceil \log \sigma / \log w \rceil + mn)$ and $O(D \lceil \log \sigma / \log w \rceil + m(n + \sigma))$ algorithms. The space occupation remains unchanged.

Another option is to set $e = w$ and organize E as an array of $\lceil n/w \rceil$ unsigned integers. To check whether a column index j is present in E , it suffices to check

⁴ It is also possible to precompute the number of leading zeros for all the integers from the range $[0, 2^{\lceil w/2 \rceil} - 1]$ in time $O(2^{\lceil w/2 \rceil})$. Thanks to the lookup table we can find the number of leading zeros in every w -bit integer in two lookups. This strategy is attractive as long as $2^{\lceil w/2 \rceil} = O(D \log \sigma / \log w + mn)$.

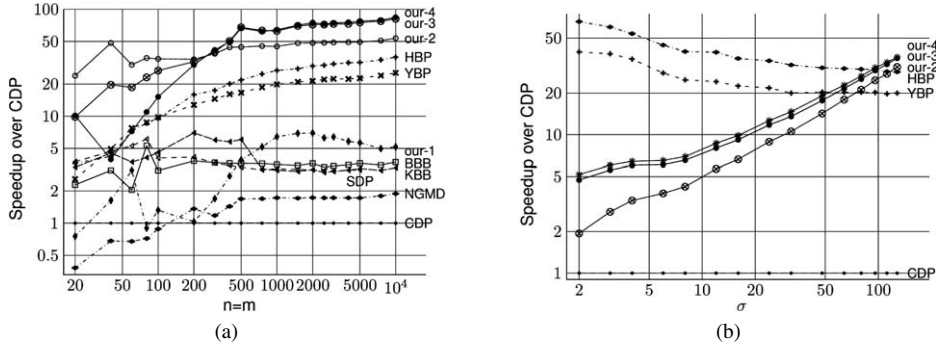


Fig. 4. Relative performance of algorithms solving the LCTS problem defined as $\text{CDP}_{\text{time}}/\text{Alg}_{\text{time}}$. (a) The dependency on the sequences length is shown. (The SDP algorithm could not be evaluated for long sequences due to its huge space consumption.) (b) The dependency on the alphabet size is shown for $n = m = 1000$.

the $(j \bmod w)$ th bit in the $\lfloor j/w \rfloor$ th integer, which needs $O(1)$ time. Setting or removing a column index is analogous. Finding the successor of $(j \bmod w)$ in the $\lfloor j/w \rfloor$ th integer, also can be done in a constant time, but there is an additional cost of unsuccessful findings of successors of $O(\lceil n/w \rceil)$ per row, $O(m \lceil n/w \rceil)$ per transposition. This leads to $O(mn + m\sigma \lceil n/w \rceil)$ algorithms. The space consumption remains unchanged.

5. Comparison of algorithms for LCTS

To compare the LCTS algorithms in practice we made two experiments. In the first one, we measured their speed on pitch sequences from real MIDI files ($\sigma = 127$). This makes the experiments different from presented by Lemström et al. [8] since they used randomly produced sequences over the alphabet $[0, 127]$. We assumed $n = m$ and the sequence lengths varied from 20 to 10 000. We compared here our four algorithms: our-1 of complexity $O(D \log \log \sigma + mn)$, our-2 of complexity $O(D \lceil \log \sigma / \log w \rceil + mn)$, the space efficient our-3 of complexity $O(D \lceil \log \sigma / \log w \rceil + m(n + \sigma))$, and the space efficient our-4 of complexity $O(mn + m\sigma \lceil n/w \rceil)$ to the algorithms mentioned in Section 3.

The experiments were performed on an AMD Athlon 2500 XP+ machine (1800 MHz real CPU clock) with 512 MB RAM. The source codes for the BBB, CDP, KBB, SDP, YBP, HBP algorithms come from their authors, and the implementation of the NGMD was made by ourselves. The routines were optimized to achieve better speed and similar optimization level as our implementations. The running times are median values of 501 executions for different pairs of sequences. The results are shown in Fig. 4(a), where we present the relative performance of the algorithms, while the reference is the CDP method. We decided to show the relative performance rather than absolute times since the lowest and

the highest times differ by 8 orders of magnitude for the tested range of sequences and the absolute-time graph would be illegible.

As we can see, HBP is the fastest algorithm given in literature. This is an LCS method executed $2\sigma + 1$ times for each possible transposition. The our-1 and NGMD algorithms are slow. The our-2 algorithm runs several times faster for short sequences ($0 < n, m < 500$) and about 50% faster for long ones ($1000 < n, m \leq 10000$) than HBP. Even faster (twice as fast as HBP), for sequences longer than 300 are our-3 and our-4. They are of worse time complexity, but win probably due to their simplicity. Interestingly, although they solve the LCTS problem they are only 3 times slower than pure CDP for LCS for $n = m = 10000$.

The standard deviations of the running times of algorithms are also interesting. For $n = m = 10000$ the ratio $100\% \times \text{std_dev}/\text{median}$ is: 26.5% for BBB, 0.8% for CDP, 0.5% for HBP, 24.4% for KBB, 1.2% for NGMD, 7.9% for SDP ($n = m = 5000$), 0.6% for YBP, 18.4% for our-1, 6.4% for our-2, 8.5% for our-3, 7.7% for our-4.

In the second experiment, we evaluated the impact of the alphabet size on the speed of the algorithms. We chose here only the fastest algorithms from the previous experiments. The sequences are randomly generated over the $[0, \sigma]$ alphabet, while the distribution is uniform. The results are shown in Fig. 4(b). We can see that our algorithms clearly lose when the alphabet size is small and win when it is large. The relative performance of the YBP and HBP methods compared to the CDP method get worse when the alphabet size grows, while for our methods it gets better. It can be also noticed that for $\sigma = 127$ the difference between HBP and our-4 is smaller than in the first experiment. This is probably caused by the artificial data used in the second experiment.

6. Conclusions

We introduced a family of algorithms for computing the longest common transposition-invariant subsequence length. The first of them achieves the $O(D \log \log \sigma + mn)$ worst-case time complexity, which improves the currently best known algorithm [14], and is significantly simpler. The second algorithm is a more practical variant of the previous one. It offers complexity $O(D \lceil \log \sigma / \log w \rceil + mn)$, which for typical values $\sigma = 127$, $w = 32$ is attractive. It is faster in the whole examined range of sequence lengths than all algorithms known from literature. The space complexity of these methods is $O(m\sigma)$ which is usually worse than $O(\sigma^2 + m)$ of the Navarro et al. proposal [14].

We also presented three space efficient variants of space complexity $O(n + \sigma)$ and time complexities $O(D \log \log \sigma + m(n + \sigma))$, $O(D \lceil \log \sigma / \log w \rceil + m(n + \sigma))$, and $O(mn + m\sigma \lceil n/w \rceil)$. The last two mentioned algorithms are the fastest in the real-data experiment for long sequences, i.e., $300 < n, m \leq 10\,000$. (It is interesting also that they solve the LCTS, but they are only 3 times slower, for $n = m = 10\,000$, than pure CDP for LCS.) They are also the fastest for artificial data for large alphabets. For small alphabets, $\sigma < 96$, the HBP [6] algorithm is the fastest.

The presented algorithms compute the length of the $LCTS(A, B)$. Since a transposition-invariant *indel* distance is a dual of this length it can be found easily: $id^t(A, B) = n + m - 2|LCTS(A, B)|$. Unfortunately, the algorithms are not easy to adapt to more complex distances, like the Levenshtein distance [5].

We discussed only how to find the LCTS length, not the subsequence itself. It is, however, possible to compute also this subsequence. To this end, we need an additional matrix T of size $m \times n$. In each cell $T[i, j]$, we store a pair $\langle i', j' \rangle$ denoting the highest-rank match in $D[0..i - 1, 0..j - 1]$ according to transposition $t = A_i - B_j$. (This can be easily combined with any of the presented algorithms.) After finishing the algorithm, we will be able to trace back the sequence using the T matrix.

Acknowledgements

The author thanks Szymon Grabowski, Gonzalo Navarro, Roman Starosolski and the anonymous re-

viewers for reading the preliminary versions of the paper and suggesting important improvements. The source codes by Yoan Pinzon and Heikki Hyyrö were also helpful.

References

- [1] L. Bergroth, H. Hakonen, T. Raita, A survey of longest common subsequence algorithms, in: Proc. of SPIRE, 2000, pp. 39–48.
- [2] M. Crochemore, C.S. Iliopoulos, Y.J. Pinzon, Speeding-up Hirschberg and Hunt–Szymanski LCS algorithms, in: Proc. of SPIRE, 2001, pp. 59–67.
- [3] T. Crawford, C. Iliopoulos, R. Raman, String matching techniques for musical similarity and melodic recognition, Computing in Musicology 11 (1998) 71–100.
- [4] M. Crochemore, G.M. Landau, M. Ziv-Ukelson, A sub-quadratic sequence alignment algorithm for unrestricted cost matrices, SIAM Journal on Computing 32 (6) (2003) 1654–1673.
- [5] D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press, Cambridge, MA, USA, 1999.
- [6] H. Hyyrö, Bit-parallel LCS-length computation revisited, in: Proc. of the 15th Australasian Workshop on Combinatorial Algorithms (AWOCA 2004).
- [7] J.W. Hunt, T.G. Szymanski, A fast algorithm for computing longest common subsequences, Communications of the ACM 20 (5) (1977) 350–353.
- [8] K. Lemström, G. Navarro, Y. Pinzon, Practical algorithms for transposition-invariant string-matching, Journal of Discrete Algorithms 3 (2–4) (2005) 267–292.
- [9] K. Lemström, J. Tarhio, Searching monophonic patterns within polyphonic sources, in: Proc. of RIAO'2000, pp. 1261–1279.
- [10] K. Lemström, E. Ukkonen, Including interval encoding into edit distance based music comparison and retrieval, in: Proc. of AISB'2000 Symposium on Creative & Cultural Aspects and Applications of AI & Cognitive Science, 2000, pp. 53–60.
- [11] V. Mäkinen, G. Navarro, E. Ukkonen, Algorithms for transposition invariant string matching, in: Proc. of STACS'2003, in: Lecture Notes in Computer Science, vol. 2607, Springer, Berlin, 2003, pp. 191–202.
- [12] V. Mäkinen, G. Navarro, E. Ukkonen, Transposition invariant string matching, Journal of Algorithms 56 (2) (2005) 124–153.
- [13] G. Navarro, A guided tour to approximate string matching, ACM Computing Surveys 33 (1) (2001) 31–88.
- [14] G. Navarro, Sz. Grabowski, V. Mäkinen, S. Deorowicz, Improved time and space complexities for transposition invariant string matching, Technical Report TR/DCC-2005-4, University of Chile, Department of Computer Science, <ftp://ftp.dcc.uchile.cl/pub/users/gnavarro/mnloglogs.ps.gz>.
- [15] P. van Emde Boas, R. Kaas, E. Zijlstra, Design and implementation of an efficient priority queue, Mathematical Systems Theory 10 (1977) 99–127.
- [16] P. van Emde Boas, R. Kaas, E. Zijlstra, Preserving order in a forest in less than logarithmic time and linear space, Information Processing Letters 6 (3) (1977) 80–82.